

Two-Factor RFID + Face Recognition Door

High-Level System Design & Architecture

Document ID

DOC-02

Status

Implementation Baseline

Design objective: Keep one authoritative local control loop, isolate hardware and ML behind ports, and preserve future expansion seams without introducing distributed infrastructure into the MVP.

1. Architecture Overview

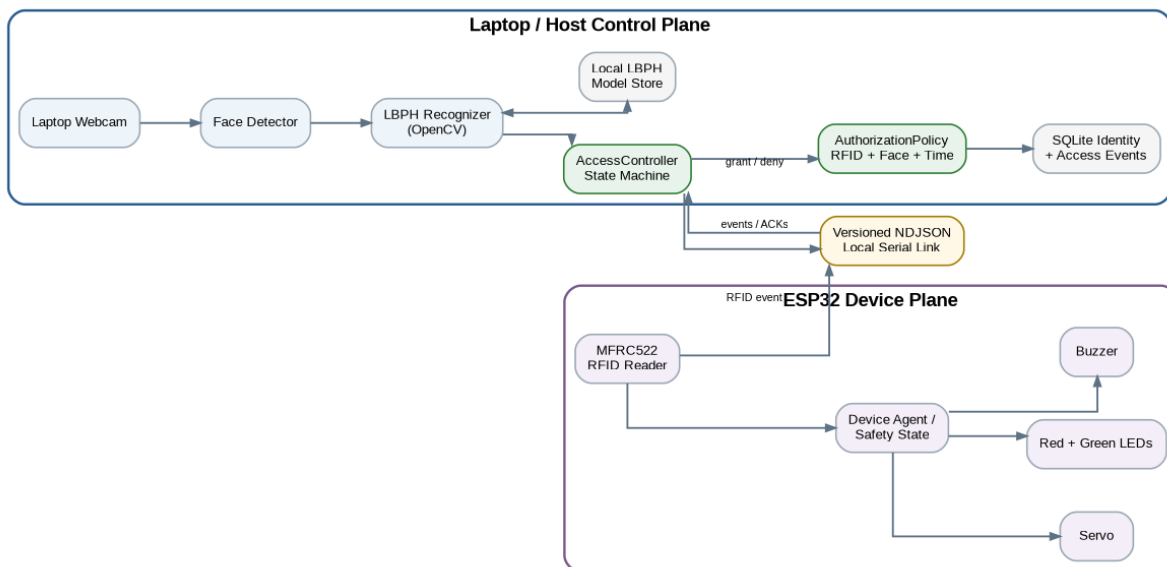


Figure 1. High-level component architecture

The laptop is the system brain. The ESP32 is a deterministic device agent. The core application makes the access decision; adapters execute the decision on hardware. This avoids coupling recognition code directly to servo control.

2. Architectural Principles

Principle	Decision	Reason
Single decision authority	AccessController + AuthorizationPolicy	Prevents an RFID handler, ML module, or device adapter from independently opening the door.
Local first	SQLite + local files + USB serial	Matches the real MVP: one door, one host, few users, no internet requirement.
Ports and adapters	CameraPort, FaceRecognizerPort,	Allows later technology changes

Principle	Decision	Reason
	DeviceTransportPort, repositories	without rewriting the domain policy.
Fail closed	Safe states at host and device	Physical access control must not open because of uncertainty or failure.
Explicit state machine	AccessSession state is persisted/logged in events	Makes timing, failure, and test behavior deterministic.
Scalability by seam	Device ID, repository interface, transport abstraction	Future scale becomes an extension of an existing boundary, not a rewrite of business logic.

3. Logical Layers

Layer	Responsibilities	Must not do
Domain	AccessSession, Identity, AccessDecision, ReasonCode, AuthorizationPolicy	Call OpenCV, serial, camera, filesystem, GPIO.
Application	AccessController, EnrollmentService, HealthService, EventLogger	Own low-level hardware details.
Ports	Interfaces for RFID, camera, recognizer, transport, clock, persistence	Contain implementation-specific behavior.
Adapters	OpenCV, SQLite, serial, filesystem, clock, model store	Make business authorization decisions.
ESP32 firmware	RFID I/O, actuator control, command validation, device-safe state	Recognize faces or decide identity authorization.

4. Runtime Interaction

1. ESP32 detects a card and emits rfid_detected with a unique event_id.
2. DeviceTransportAdapter delivers the event to AccessController.
3. AccessController resolves card ownership through IdentityRepository.
4. A valid card creates AccessSession(session_id, expected_person, deadline).
5. CameraPort provides frames while the session is VERIFYING.
6. FaceRecognizerPort produces RecognitionResult(label, distance) for detected faces.
7. AuthorizationPolicy evaluates identity binding, threshold, match stability, and time.
8. Only a GRANT decision is translated into an access_command to the device.
9. ESP32 validates and executes the command, then returns command_result.
10. EventLogger records the outcome and correlation identifiers.

5. Failure Handling Matrix

Failure	Host action	Device action	Result
Unknown RFID	Do not start face verification; log UNKNOWN_RFID	Remain closed	DENIED
No face	Keep scanning until deadline	No output change	TIMEOUT if deadline expires
Wrong enrolled face	Immediate deny	Red + long buzz; closed	DENIED
Multiple faces	Treat as ambiguous and deny	Red + long buzz; closed	DENIED
Low-confidence/threshold fail	Continue within 10s unless explicit wrong identity is known	No grant	Eventually TIMEOUT/DENIED
Camera failure	Enter FAULT; stop grant path	Remain closed	FAULT
Serial loss	Cancel/deny outstanding authorization	Remain closed	FAULT/DENIED
Malformed command	Log and reject	Ignore safely	No grant
Duplicate grant command	Idempotent by command_id	Do not re-trigger open unnecessarily	Safe
Restart	Reinitialize into CLOSED/IDLE	Servo closed target, indicators safe	Safe recovery

6. Authorization State Model

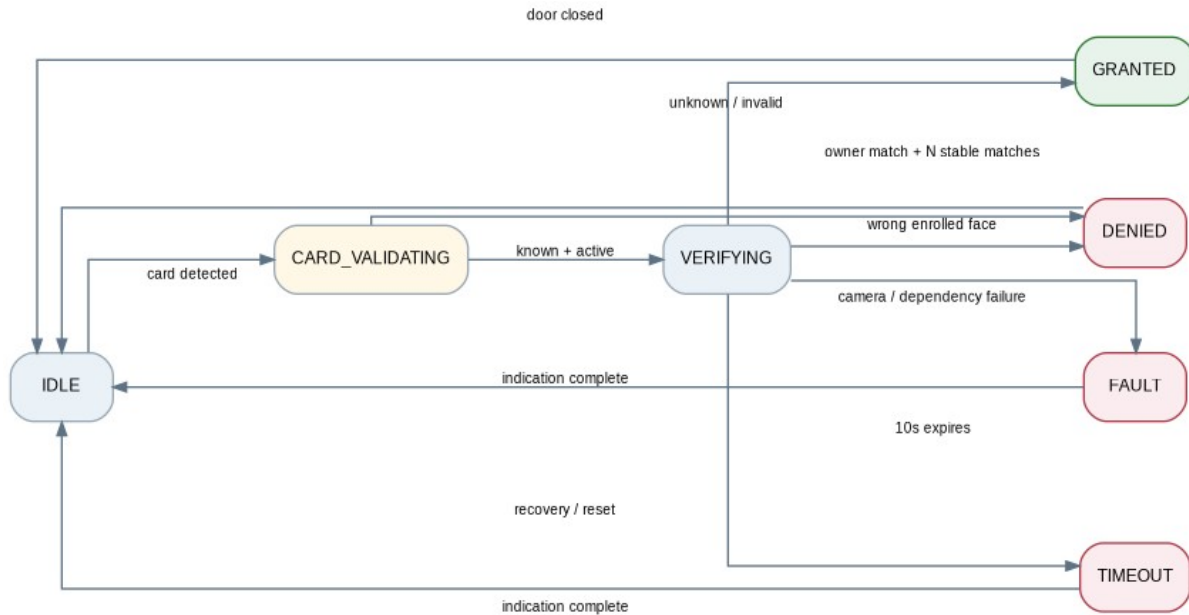


Figure 2. State ownership is centered in the host controller

7. Communication Boundary

For the MVP, use the USB serial path exposed by the ESP32 development board. The host sees a logical DeviceTransportPort, so the implementation can later switch to a network transport without changing the domain or authorization modules. Espressif documents configurable UART parameters and application use of UART interfaces.

```

TransportPort
  send(command: DeviceCommand) -> None
  receive() -> DeviceEvent | None
  health() -> TransportHealth

Message envelope
  v: integer
  type: string
  event_id/session_id/command_id: correlation identifiers
  payload: object
  
```

8. Scalable Boundary Map

MVP implementation	Future replacement without domain rewrite
SQLiteIdentityRepository	PostgreSQLIdentityRepository / remote identity service
Local JSONL/FileEventRepository	Append-only event store / centralized audit pipeline
OpenCVCameraAdapter	Network camera / video capture service
OpenCVLBPHRecognizer	Another local model or isolated inference service

MVP implementation	Future replacement without domain rewrite
Esp32SerialAdapter	Authenticated network device gateway
Single DeviceRegistry entry	Multiple-door device registry

Scale rule: Do not introduce a microservice until a real boundary requires independent deployment, ownership, scaling, or trust. The ports above are the seams that make such evolution possible later.

9. Data Ownership

Data	Owner	Storage
Person and RFID bindings	IdentityRepository	SQLite
Face training samples	EnrollmentService	Local filesystem
LBPH model + label map	FaceRecognizer adapter / ModelStore	Local filesystem
Access events	EventLogger / EventRepository	SQLite + optional JSONL export
Live session state	AccessController	In-memory; events persisted for audit
Physical device state	ESP32 Device Agent	ESP32 RAM; reported by heartbeat/status

10. Observability

- Every RFID event gets event_id.
- Every authorization attempt gets session_id.
- Every device action gets command_id.
- Logs include reason codes instead of free-form only messages.
- Heartbeat exposes device_id and physical safe state.
- Recognition diagnostics may log distance/threshold for development, but raw face images should not be copied into routine access logs.

11. Deployment Topology

MVP deployment

Laptop

- ├─ access-control host process
- ├─ local SQLite database
- ├─ local face samples / LBPH model
- └─ USB serial connection



12. Hardware Design Notes

A practical baseline pin map is given in the LLD. The MFRC522 is a 13.56 MHz contactless reader/writer IC and NXP currently marks the MFRC522 product family as End of Life / not recommended for new designs. It is therefore acceptable here as a fixed educational prototype component, but it should not be used as the basis for a new commercial hardware platform without lifecycle review.

Power integrity: Do not power a servo directly from a fragile logic rail. Use an appropriate 5V supply for the servo, tie grounds together, and keep the MFRC522 logic at its supported voltage. Poor servo power can reset the ESP32 and create misleading software failures.

13. Technology Choices

Area	MVP choice	Why
MCU	ESP32	Low-cost control of RFID and simple actuator I/O.
RFID	MFRC522	Fixed user requirement; simple reader integration for prototype.
Host language	Python	Fast local iteration and direct OpenCV integration.
Computer vision	OpenCV + LBPH	Explicit project requirement; deterministic, local, understandable.
Persistence	SQLite	Enough for a single local host and small user population.
Protocol	NDJSON over USB serial	Simple, debuggable, versionable; no broker/server required.